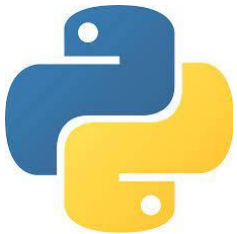
Auto Insurance Fraud Detection



(Machine Learning Project)



Prepared By : Ramchandra Babar

Project Introduction

Problem Statement:

To build a classification methodology to determine whether a customer is placing a fraudulent insurance claim.

- Problem can be categorized as supervised learning anomaly detection of unbalanced data.

Project Introduction

Problem Description:

The insurance claims are made by client and system has to identify whether certain client making fraudulent claim or not. Whenever their is any damage or breakage to vehicle property in an accident, **people claim through an insurance** but even if their is no damage ie **normal wear & tear** as well people want to claim **with false proof.**

So to curve this issue, insurance **company want an application** which will able to **correctly identify fraudulent** & genuine claims. The fraudulent claim will goes through manual inspection & based on facts decide whether claim is genuine or not. So, we have to build a model that can detect this auto insurance fraud **to reduce losses** to the company.

Understanding Data

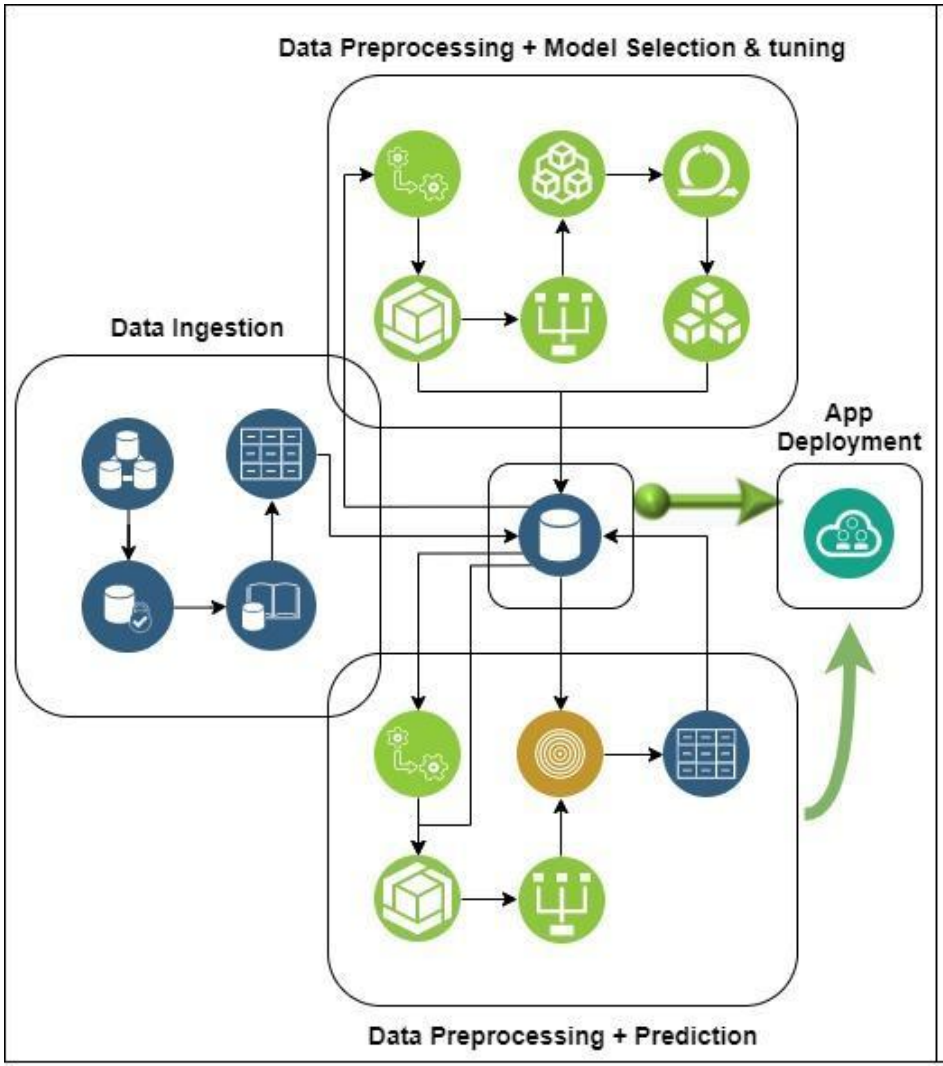


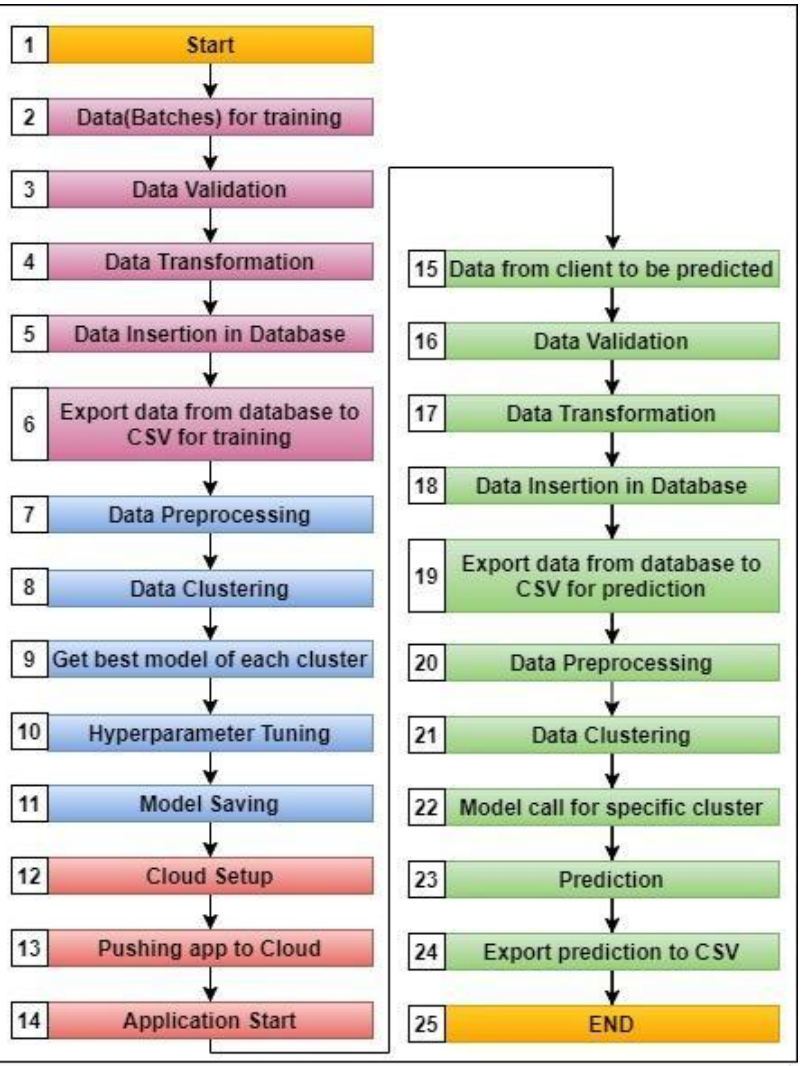
No. of Records: 1000 & No. of Features/Columns: 38

Features in Data

Category	Features		
Insurance Details	months_as_customer	age	policy_number
	policy_bind_date	policy_state	policy_csl
	policy_deductable	policy_annual_premium	umbrella_limit
	insured_zip	insured_sex	insured_education_level
	insured_occupation	insured_hobbies	insured_relationship
	capital-gain	capital-loss	
Accident Details	incident_date	incident_type	collision_type
	incident_severity	authorities_contacted	incident_state
	incident_city	incident_location	incident_hour_of_the_day
	property_damage	bodily_injuries	Witnesses
	police_report_available	total_claim_amount	injury_claim
	property_claim	vehicle_claim	auto_make
	auto_model	auto_year	
Target Label	fraud_reported (Y/N)		

Architecture & Application Flow





Data Ingestion

Data Validation

- The Client is sending the data in batches keeping them into file location or network shared address.
- Received batch files validated through pipeline using **JSON schema file** which contains file properties validation rules based on NDA.
- We validate text in **File Name** using regex pattern & check length of date & time using if condition.
- Simillarly, in file we validate the **Number of Columns, Name of Columns, Datatype of Columns, Null Values in Columns** & Files not fulfilling requirement as per schema archived to "Bad_Data_Folder"
- If any file has invalid data type in any of the columns, the file is not loaded in the table and is moved to "Bad_Data_Folder".

Data Ingestion

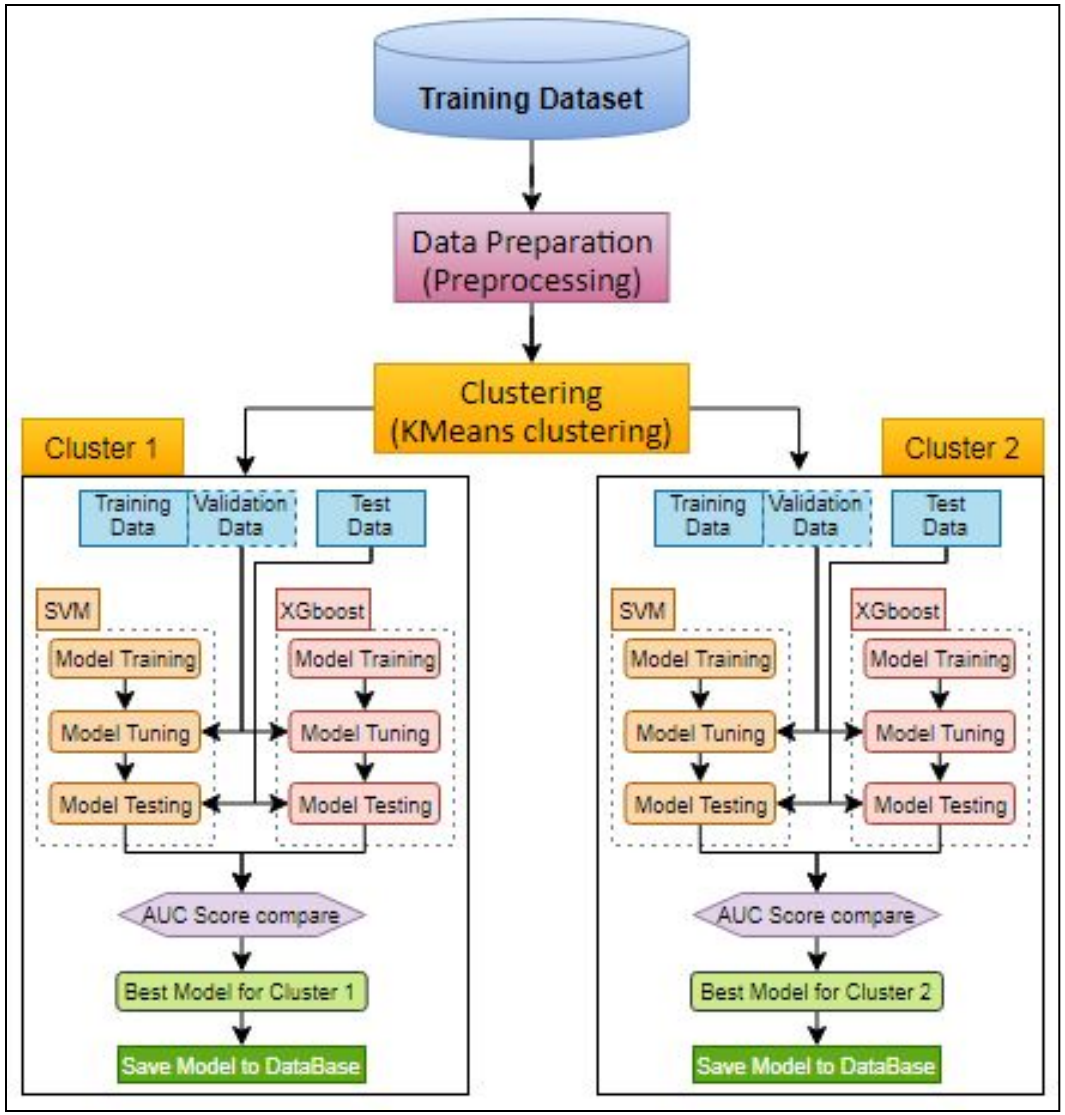
Data Transformation

- Files fulfilling all requirement passed to data transformation pipeline where we replace missing values with null, single quote replace by double quote & moved to "Good_Data_Folder”

Data Insertion in Database

- Instead of building model for individual files we agreegate all files.
- Create a database if not already present & connect to the database.
- In database if table "Good_Data" not present then create table with that name. All the files in the "Good_Data_Folder" are inserted in the above-created table.
- The data in a stored database is exported as a CSV file to be used for model training.

Model selection & tuning



Model selection & tuning

Data Preprocessing

- Selection of features or columns required for prediction.
- For this dataset, ‘?’ have been replaced with NaN values.
- If data contain null/NaN values impute it using categorical imputer.
- Replace & encode the categorical values with numeric values.
- Scale the numeric values using the standard scaler.
- Balance unbalanced data or Normally distribute data (based on algorithm using in training)

Model selection & tuning

Clustering

- Instead of creating single model for whole data we can divide data into clusters & train different models on individual cluster which gives better model performance.
- For clustering we can use KMeans algorithm & to decide number of clusters "KneeLocator" function. The Kmeans model is trained over preprocessed data & model is saved for further use in prediction.

Model Selection

- After creating clusters, for each cluster we find the best model out of "SVM" and "XGBoost" with best parameters using GridSearchCV hyper-parameter tunning.
- Best models selected based on AUC scores.
- All the models for every cluster are saved for use in prediction.

Prediction

Data Validation & Transformation

- The prediction file will not contains labels, which we have to find.
- After getting files for prediction, we validate as per schema file & based on verification results pass to Good_Data or Bad_Data in database then exported to CSV to be used for prediction.

Data Preprocessing

- Here same steps followed as done with training files.

Clustering

- Here we apply saved Kmean clustering algorithm or model to get cluster number for given row or record then we segregate all data based on cluster numbers.

Prediction

Prediction

- Based on the cluster number, the respective model of cluster is loaded for prediction.
- Once the prediction is made for all the clusters, the predictions along with input are saved in a CSV file at a given location, and the location is returned to the client.

App Deployment

- In Cloud setup, we prepare prerequisite file so that application run smoothly.
  1. **requirements.txt** : required python packages to run application
  2. **runtime.txt** : specify which version of python to be used
  3. **Procfile** : Process file specify entry point to cloud application

     Eg. web: gunicorn main: app

     It tell heroku to use the app variable (constructed flask app) in the main module with gunicorn & to start a web process (one that can handle http requests).

- Then we push app code to cloud & cloud platform itself will start application & keep it running.
- For Demo we have deployed developed WebApp using **Heroku** which provide Platform as a Service.

Project Codebase & Platforms

1. Kmean ⟹ **Clustering Data**
2. SVM & XGboost ⟹ **Model Training**
3. Flask ⟹ **WebApp UI**
4. Heroku ⟹ **Deployment**

❖Modular Programming Approach:

- Dividing program into separate sub-programs

•Created different modules for each task in Lifecycle.

- **Benefits:** Code easier to debug, Reusable Code, Increased Readability & Manageability.

Modules in Project

Sr No	Module / Pipeline	Purpose	Code
1	main.py	**Entry point for App with flask**	*link*
2	data_loader, data_loader_prediction	**Read the Data from source**	*link*
3	rawValidation	**Raw Data Validation in Training based on schema**	*link*
4	DataTypeValidation	**Validation of DataType or Training Data**	*link*
5	DataTransformation	**Data Transformation for Training Data**	*link*
6	clustering, preprocessing	**Preprocessing of data before Model training**	*link*
7	trainingModel.py	**From Training data form cluster & build best model**	*link*
8	tuner	**Fine tuning of model**	*link*
9	logger	**Log required tasks**	*link*
10	predictionDataValidation	**Raw Data Validation in Prediction based on schema**	*link*
11	DataTypeValidationPrediction	**Validation of DataType for Prediction Data**	*link*
12	DataTransformationPrediction	**Data Transformation for Prediction Data**	*link*
13	predictFromModel.py	**Predict results based on input & model**	*link*

main.py

- Entry point for application is main.py which contains modules or routes created using flask app. Here control will be transferred according to route name is getting called.
- There are three specified routes.

1) Home Route 2) Training Route 3) Prediction Rou**te**

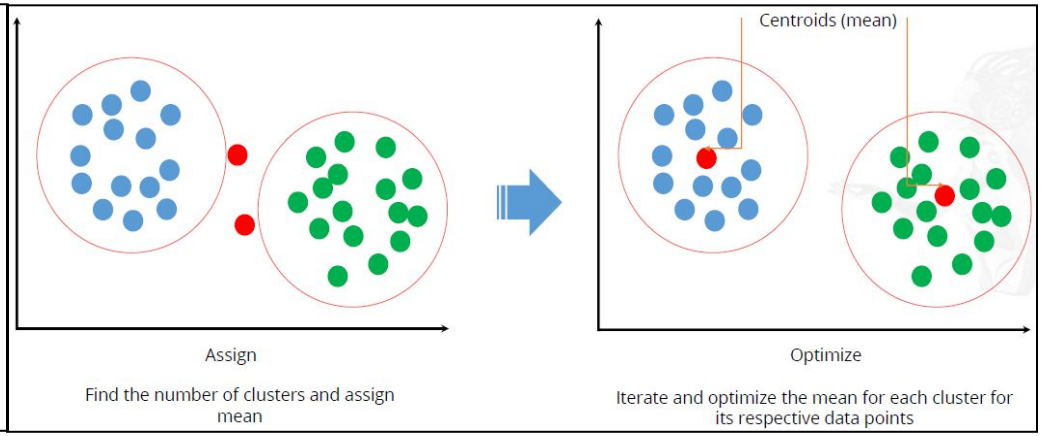
1. **Home route:** It is application home page ie webUI or GUI for prediction. Home route just calls home page of an application.
2. **Training route:** It get called when user gives location of file which perform validation, preprocessing, clustering, ML training, saving individual cluster model.
3. **Prediction route:** In prediction route control goes to file location which perform validation, preprocessing, clustering to get cluster number, segregate data based on cluster, prediction from individual cluster model, prediction aggregated & saved to CSV file which is final output of prediction model.

K-mean Clustering

- A cluster is a collection or group of “similar” data points.
- The numbers of clusters needs to know beforehand & decided based analysis of data by considering business priorities or constraints.
- K-means finds the best centroids by alternatively assigning random centroids to a dataset and selecting mean data points from the resulting clusters to form new centroids. It continues this process iteratively until the model is optimized.

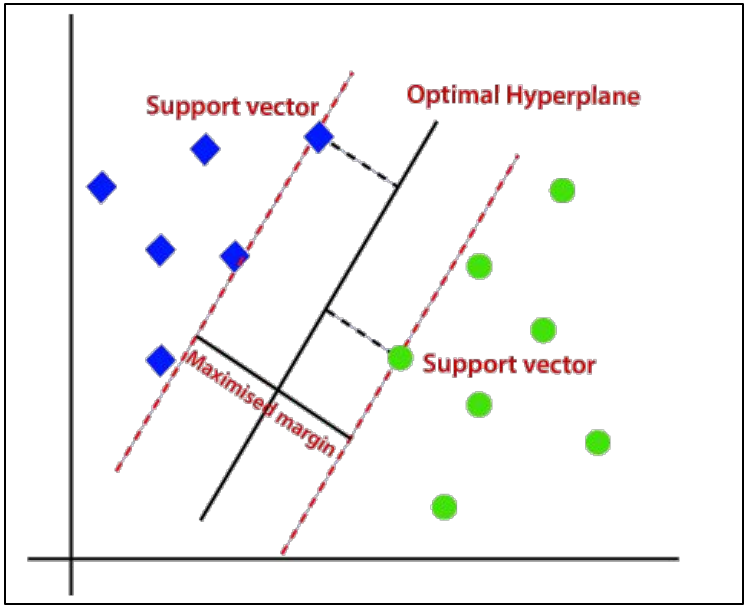
Steps:

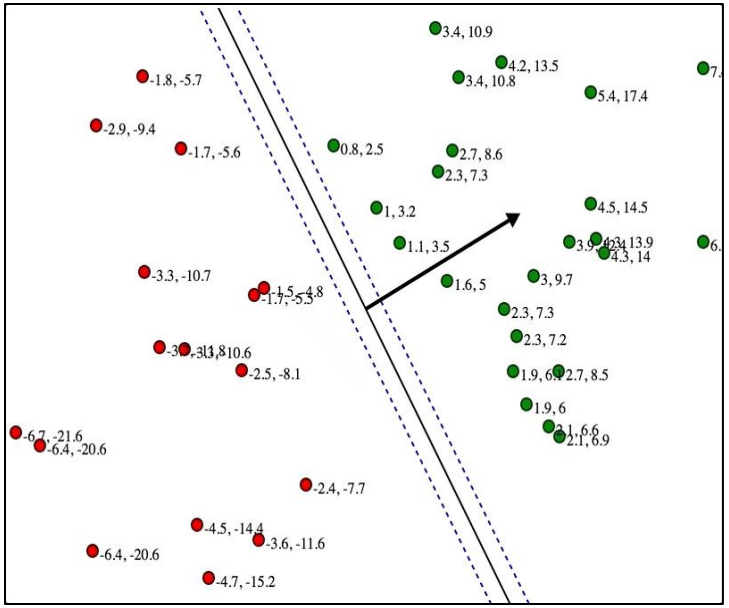
1. Choose a mean from each cluster as a centroid
2. Reassign data points to new centroids
3. Iterate the process till the model is optimized



Support Vector Machine (SVM)

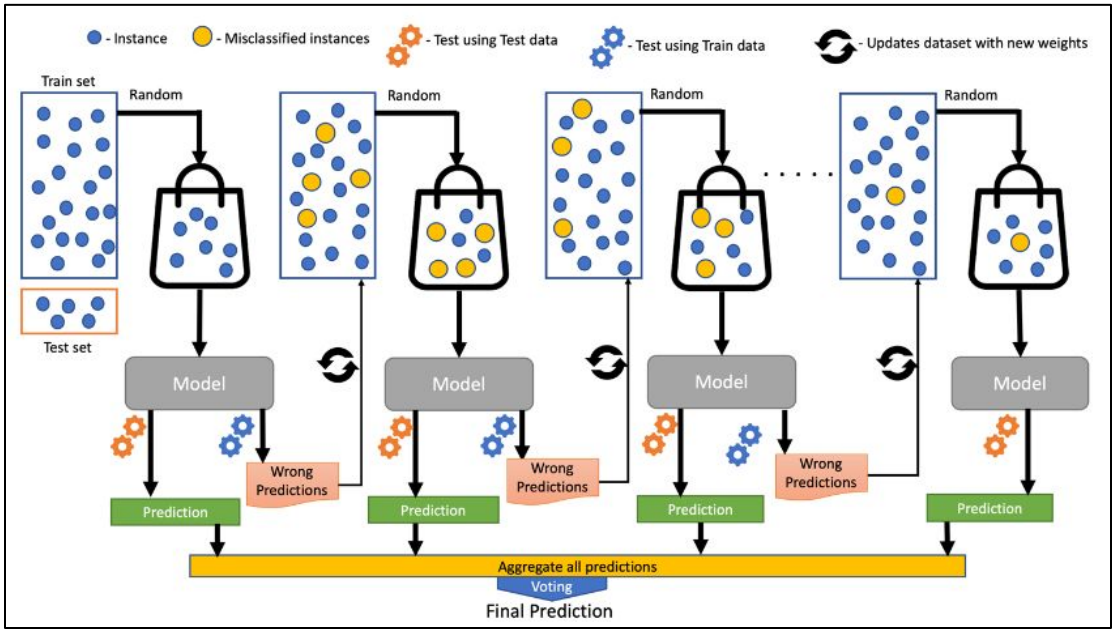
- SVM is a supervised learning non-probabilistic linear classifier which tries to divide the data using hyperplanes or decision boundary that can segregate n-dimensional space into classes and then makes the predictions.
- The best hyperplane has the largest possible margin. In creating the hyperplane SVM chooses the extreme points/vectors called as support vectors, and hence algorithm is termed as Support Vector Machine.
- SVMs work fine on sparse and unbalanced data.





XGBoost

- XGBoost (extreme gradient boosting) is decision-tree-based ensemble Machine Learning algorithm that uses a gradient boosting framework.
- It build decision trees sequentially such that each tree is built on a modified version of the original data set. (e.g. increasing weights of those data points that are harder to classify & decreasing weights on those that were easier).
- XGBoost tackles the class imbalance that can creates overfitting if not handled properly.



Thank You . . .